

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <errno.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/resource.h>
#include <time.h>

#define MAX_COMMANDS 16
#define MAX_ARGS 32
#define BUF_SIZE 1024

// Trim leading and trailing whitespace
char *trim_space(char *str) {
    while (*str == ' ' || *str == '\t') str++;
    if (*str == 0) return str;
    char *end = str + strlen(str) - 1;
    while (end > str && (*end == ' ' || *end == '\t')) end--;
    end[1] = '\0';
    return str;
}

// Parse command arguments for an individual stage
int parse_cmd_args(char *cmd_str, char **args) {
    int count = 0;
    char *token = strtok(cmd_str, " \t");
    while (token != NULL && count < MAX_ARGS - 1) {
        args[count++] = token;
        token = strtok(NULL, " \t");
    }
    args[count] = NULL;
    return count;
}
```

```

// Execute large multi-stage pipeline and monitor resource metrics
void execute_pipeline(char *line) {
    char *commands[MAX_COMMANDS];
    int num_cmds = 0;

    // Split pipeline by '|' delimiter
    char *token = strtok(line, "|");
    while (token != NULL && num_cmds < MAX_COMMANDS) {
        commands[num_cmds++] = trim_space(token);
        token = strtok(NULL, "|");
    }

    if (num_cmds == 0) return;

    struct timespec start_time, end_time;
    clock_gettime(CLOCK_MONOTONIC, &start_time);

    int pipefds[2 * (num_cmds - 1)];
    for (int i = 0; i < num_cmds - 1; i++) {
        if (pipe(pipefds + i * 2) < 0) {
            perror("pipe creation failed");
            return;
        }
    }

    pid_t pids[MAX_COMMANDS];

    for (int i = 0; i < num_cmds; i++) {
        pids[i] = fork();

        if (pids[i] < 0) {
            perror("fork failed during pipeline execution");
            return;
        }

        if (pids[i] == 0) {
            // If not first stage, redirect stdin from previous pipe read end

```

```

        if (i != 0) {
            if (dup2(pipefds[(i - 1) * 2], STDIN_FILENO) < 0) {
                perror("dup2 stdin failed");
                exit(EXIT_FAILURE);
            }
        }

        // If not last stage, redirect stdout to current pipe write end
        if (i != num_cmds - 1) {
            if (dup2(pipefds[i * 2 + 1], STDOUT_FILENO) < 0) {
                perror("dup2 stdout failed");
                exit(EXIT_FAILURE);
            }
        }

        // Close all pipe descriptors in child process
        for (int j = 0; j < 2 * (num_cmds - 1); j++) {
            close(pipefds[j]);
        }

        char *args[MAX_ARGS];
        if (parse_cmd_args(commands[i], args) == 0) {
            exit(EXIT_SUCCESS);
        }

        execvp(args[0], args);
        fprintf(stderr, "Pipeline Execution Error: Command '%s' failed: %s\n", args[0], strerror(errno));
        exit(EXIT_FAILURE);
    }
}

// Parent closes all pipe file descriptors
for (int i = 0; i < 2 * (num_cmds - 1); i++) {
    close(pipefds[i]);
}

// Wait for all child processes in the pipeline to finish

```

```

int failure_detected = 0;
for (int i = 0; i < num_cmds; i++) {
    int status;
    waitpid(pids[i], &status, 0);
    if (WIFEXITED(status) && WEXITSTATUS(status) != 0) {
        failure_detected = 1;
    }
}

clock_gettime(CLOCK_MONOTONIC, &end_time);
double elapsed = (end_time.tv_sec - start_time.tv_sec) +
    (end_time.tv_nsec - start_time.tv_nsec) / 1e9;

// Monitor resource consumption metrics via getrusage
struct rusage usage;
getrusage(RUSAGE_CHILDREN, &usage);

// Display performance metrics
fprintf(stderr, "\n--- PIPELINE PERFORMANCE REPORT ---\n");
fprintf(stderr, "Stages Executed      : %d\n", num_cmds);
fprintf(stderr, "Pipeline Wall Time   : %.4f seconds\n", elapsed);
fprintf(stderr, "User CPU Time Spent  : %ld.%06ld s\n", usage.ru_utime.tv_sec, (long)usage.ru_utime.tv_usec);
fprintf(stderr, "System CPU Time     : %ld.%06ld s\n", usage.ru_stime.tv_sec, (long)usage.ru_stime.tv_usec);
fprintf(stderr, "Max Resident Set    : %ld KB\n", usage.ru_maxrss);
fprintf(stderr, "Pipeline Status     : %s\n", failure_detected ? "FAILED (Non-zero exit)" : "SUCCESS");
fprintf(stderr, "-----\n\n");
}

int main() {
    char input_buf[BUF_SIZE];

    while (1) {
        printf("PerfPipelineShell > ");
        fflush(stdout);

        if (!fgets(input_buf, sizeof(input_buf), stdin)) break;

        input_buf[strcspn(input_buf, "\n")] = 0; // Strip newline

        if (strcmp(input_buf, "exit") == 0) break;
        if (strlen(input_buf) == 0) continue;

        execute_pipeline(input_buf);
    }

    return 0;
}

```

OUTPUT :

```

abhis@MIKHAIL-37:~$ nano pipeline_perf_shell.c
abhis@MIKHAIL-37:~$ gcc -g -Wall pipeline_perf_shell.c -o pipeline_perf_shell
abhis@MIKHAIL-37:~$ ./pipeline_perf_shell
PerfPipelineShell > cat /etc/passwd | grep -v nologin | wc -l
4

--- PIPELINE PERFORMANCE REPORT ---
Stages Executed      : 3
Pipeline Wall Time   : 0.0078 seconds
User CPU Time Spent  : 0.004456 s
System CPU Time      : 0.006344 s
Max Resident Set     : 7848 KB
Pipeline Status      : SUCCESS
-----

PerfPipelineShell > ls -l /usr/bin | grep python | sort | head -n 5
-rwxr-xr-x    1 root root          956 Dec  3  2025 pybabel-python3
-rwxr-xr-x    1 root root       7477160 Aug 20 10:41 python3.14
lrwxrwxrwx    1 root root          10 Mar 21  2026 python3 -> python3.14
lrwxrwxrwx    1 root root          31 Mar 21  2026 py3versions -> ../share/python3/py3versions.py

--- PIPELINE PERFORMANCE REPORT ---
Stages Executed      : 4
Pipeline Wall Time   : 0.0447 seconds
User CPU Time Spent  : 0.018763 s
System CPU Time      : 0.036892 s
Max Resident Set     : 8820 KB
Pipeline Status      : SUCCESS
-----

PerfPipelineShell > cat non_existent_file.txt | grep test | wc -l
cat: non_existent_file.txt: No such file or directory
0

--- PIPELINE PERFORMANCE REPORT ---
Stages Executed      : 3
Pipeline Wall Time   : 0.0075 seconds
User CPU Time Spent  : 0.020497 s
System CPU Time      : 0.043405 s
Max Resident Set     : 8820 KB
Pipeline Status      : FAILED (Non-zero exit)
-----

PerfPipelineShell > exit
abhis@MIKHAIL-37:~$ valgrind --leak-check=full --track-fds=yes ./pipeline_perf_shell
==541== Memcheck, a memory error detector
==541== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==541== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==541== Command: ./pipeline_perf_shell
==541==
PerfPipelineShell > exit
==541==
==541== FILE DESCRIPTORS: 3 open (3 inherited) at exit.
==541==
==541== HEAP SUMMARY:
==541==    in use at exit: 0 bytes in 0 blocks
==541==    total heap usage: 2 allocs, 2 frees, 2,048 bytes allocated
==541==
==541== All heap blocks were freed -- no leaks are possible
==541==
==541== For lists of detected and suppressed errors, rerun with: -s
==541== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

```